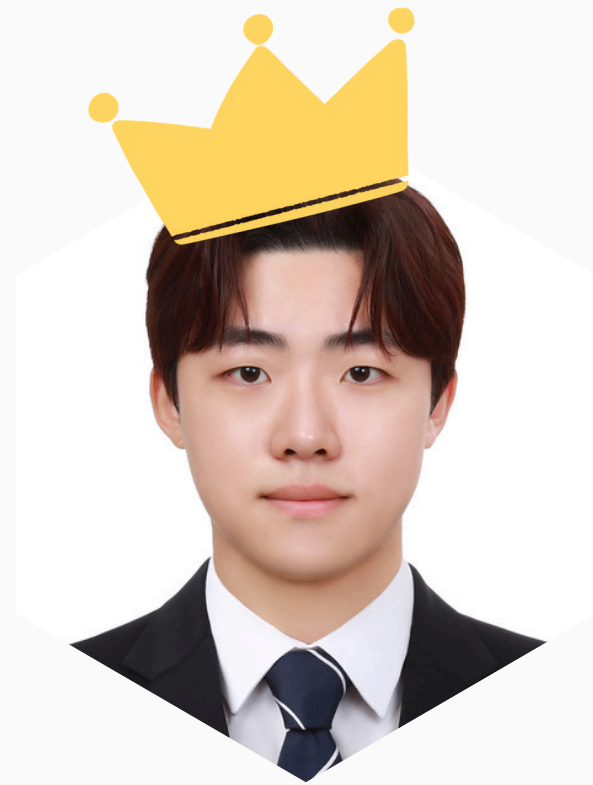


주차 보조 시스템 & 자동 출차

2조 - Overdrive

이재필, 김건우, 김재동, 김병현, 이소연, 황선안

팀원소개



이재필

팀장

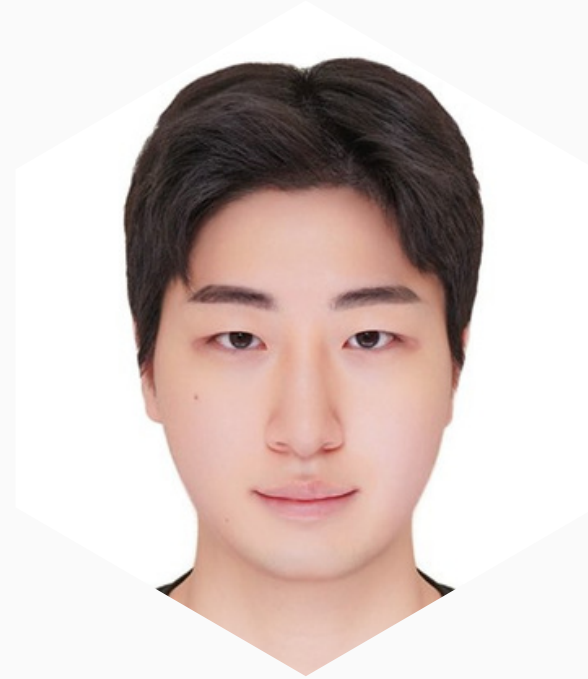
자동 출차
IMU



김재동

상무

앱
Bluetooth
IMU



김병현

계약직

CAN 통신
컨트롤러
모터ECU



김건우

대리

초음파 센서
RTOS



황선안

신입사원

라즈베리파이 통합
HMI 설계 및 구현
주차선 인식 로직



이소연

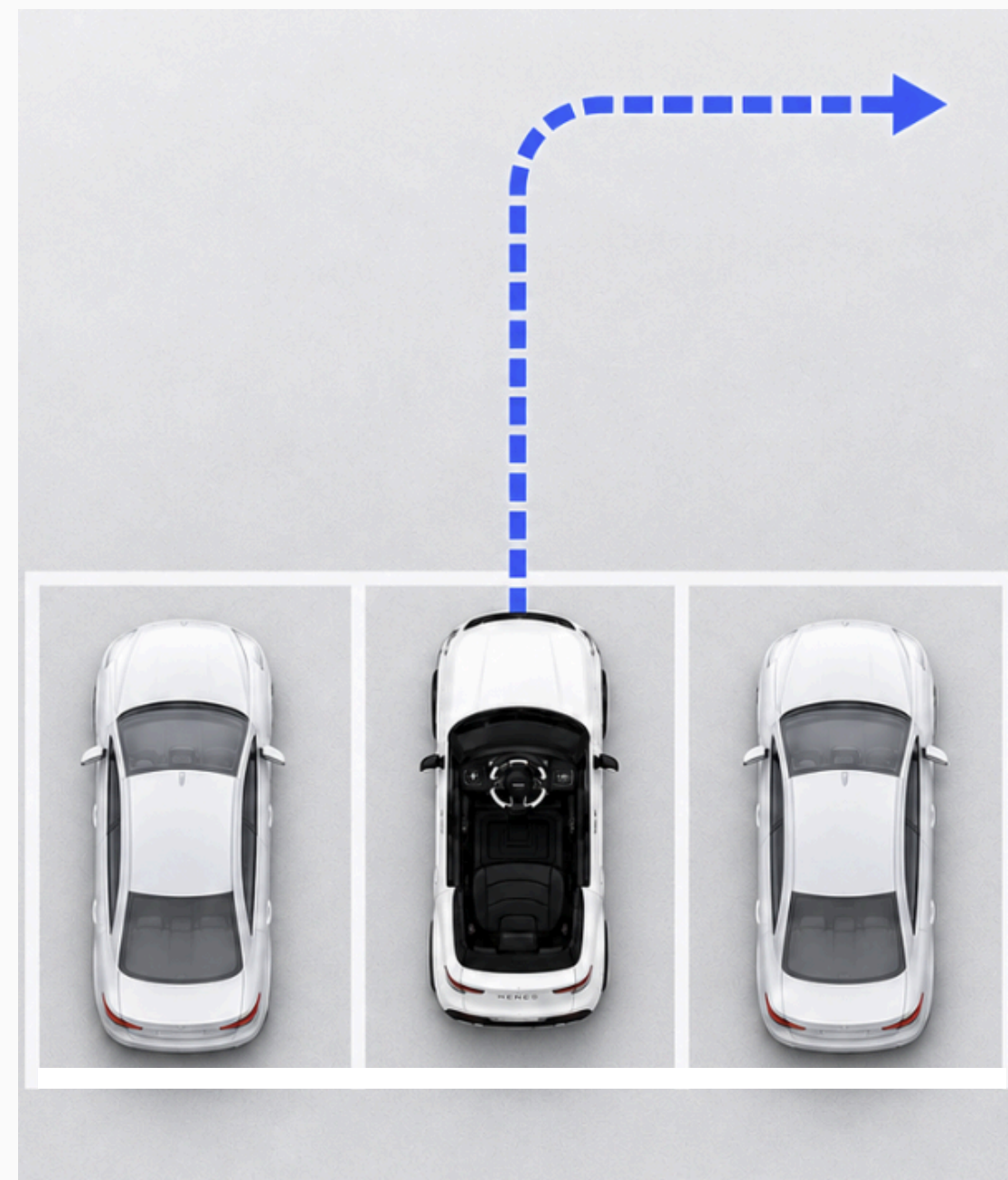
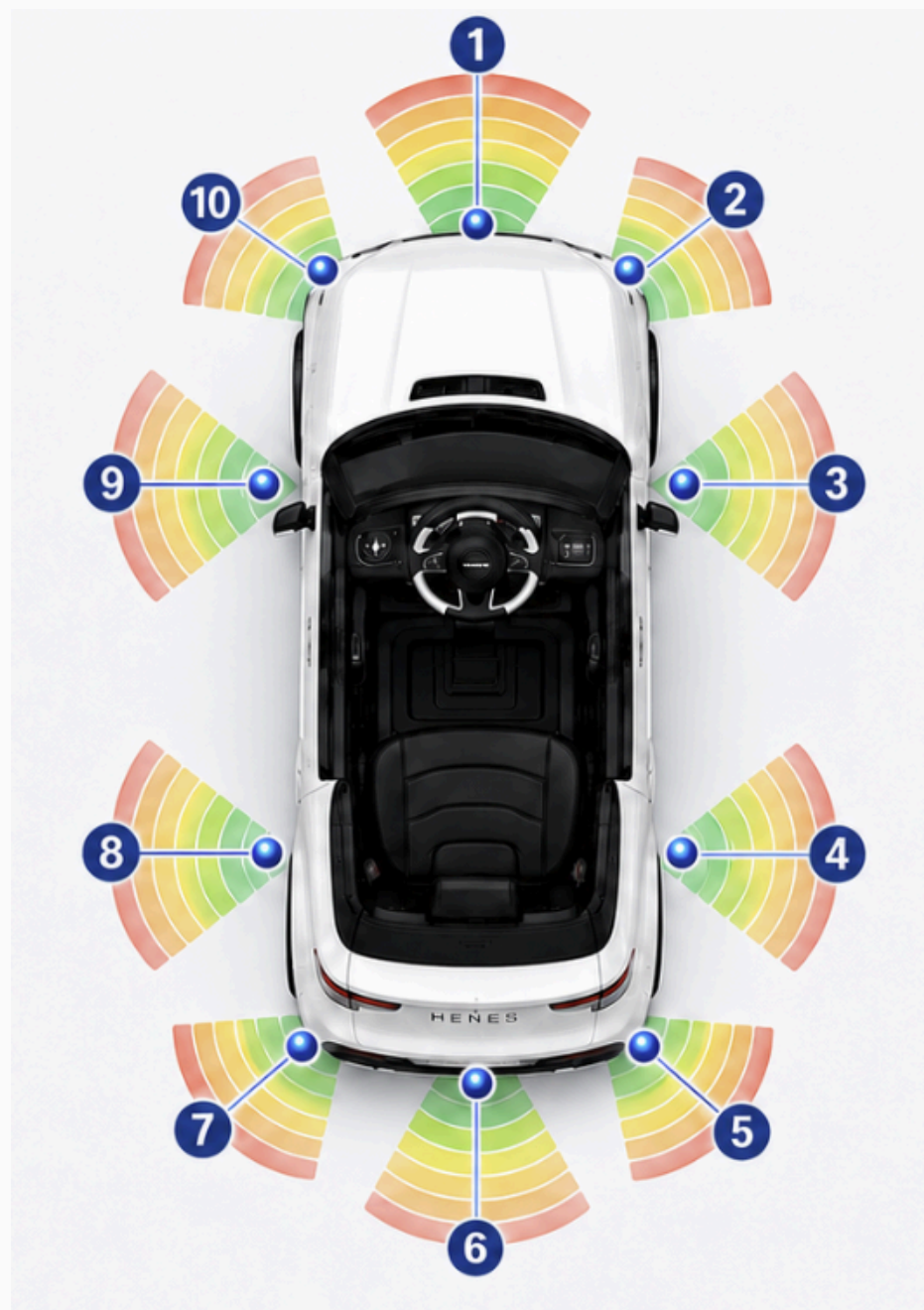
경력 인턴

PDW
RTOS
주차선 인식

Overdrive

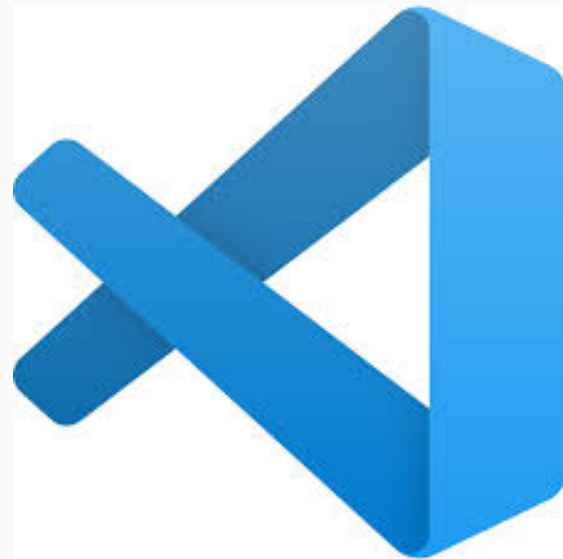
프로젝트 소개

주차 보조 시스템과 자동 출차



주차 보조 시스템과 자동 출차

01 개발



02 협업



ID	사용자 요구사항	설명
UR-001	사용자는 주차 중 차량 주변 장애물의 위험 방향을 확인할 수 있어야 한다.	시스템은 초음파 센서 정보를 기반으로 차량 주변 8방향의 장애물 위험 여부를 PDW 화면에 표시해야 한다.
UR-002	사용자는 장애물이 어느 방향에 가까운지 직관적으로 알 수 있어야 한다.	PDW 화면은 FL, FC, FR, SL, SR, RL, RC, RR의 8개 방향 구역을 기준으로 위험 방향을 표시해야 한다.
UR-003	사용자는 거리 수치가 아닌 경고 단계로 위험 정도를 인식할 수 있어야 한다.	디스플레이에는 cm, m와 같은 거리값을 직접 표시하지 않고, 색상 또는 단계 표시를 통해 위험 수준을 나타내야 한다.
UR-004	사용자는 각 방향에 따라 장애물 접근 정도를 색상 단계로 구분할 수 있어야 한다.	시스템은 장애물 없음(회색), 안전(초록), 주의(노랑), 근접(주황), 위험(빨강)과 같은
UR-005	사용자는 화면을 보지 않아도 장애물	
UR-006	사용자는 차량이 장애물에 충돌 직전	
UR-007	사용자는 주차 충돌방지 보조 기능을	
UR-008	사용자는 주차 시 후방 사각의 위험	

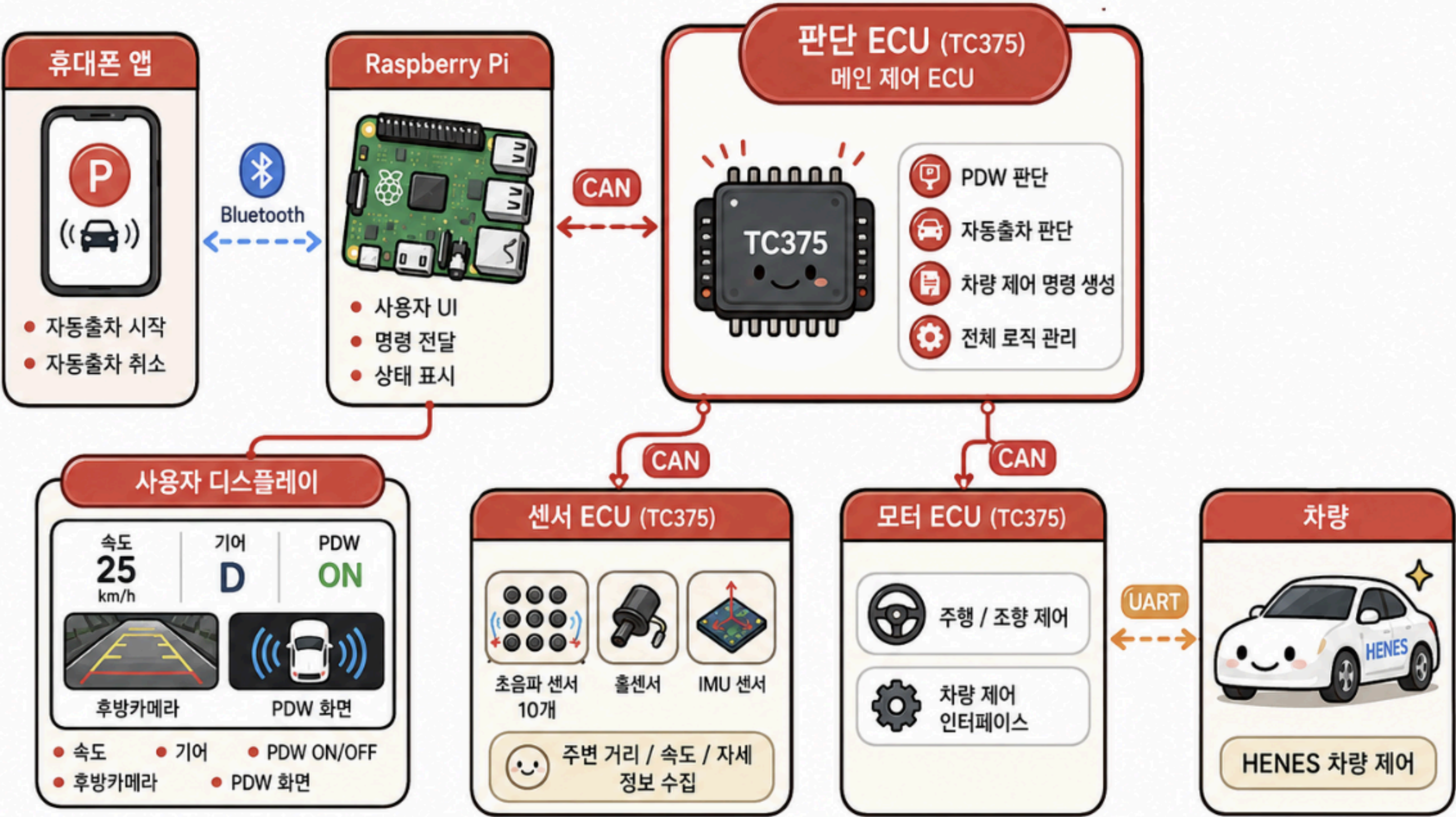
UR-001 사용자는 주차 중 차량 주변 장애물의 위험 방향을 확인할 수 있어야 한다.

사용자 요구사항 ID	요구사항 ID	파생 요구사항
UR-001	REQ-FUNC-001	시스템은 초음파 센서를 이용하여 차량 주변 장애물을 감지해야 한다.
UR-001	REQ-FUNC-002	시스템은 감지된 장애물 정보를 차량 주변 방향 정보로 변환해야 한다.
UR-001	REQ-FUNC-003	시스템은 차량 주변 장애물 위험 방향을 PDW 화면에 표시해야 한다.
UR-001	REQ-NFUNC-001	PDW 화면은 사용자가 차량 주변 장애물의 위험 방향을 직관적으로 인식할 수 있도록 구성되어야 한다.
UR-001	REQ-NFUNC-002	장애물 감지 정보는 주차 및 출차 상황에서 참고할 수 있을 정도로 안정적으로 갱신되어야 한다. → 장애물 감지 정보는 100ms 마다 최신 정보로 갱신되어야 한다

Overdrive

기능

전체 구조도 및 기능



CAN 통신 컨트롤러 & 모터ECU

CAN 통신 & 컨트롤러를 이용한 차량 제어 구현

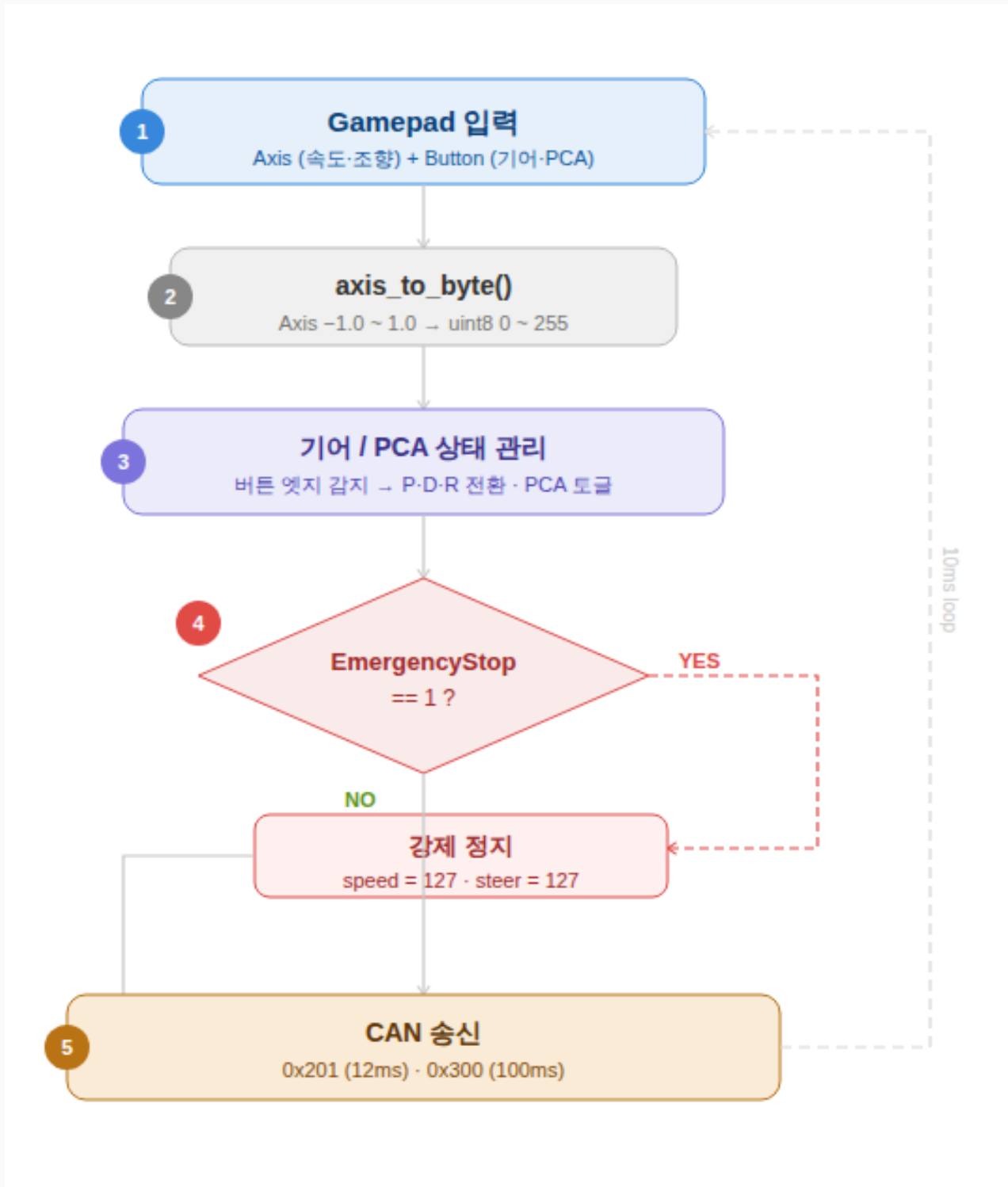
CAN 인터페이스 설계서

자율 주차 보조 시스템 — 센서 ECU · 판단 ECU · 차량 제어 ECU · Raspberry Pi

● 라즈베리파이 ● 판단 ECU ● 센서 ECU ● 차량 제어 ECU

메시지 목록

CAN ID	메시지 이름	송신 ECU	방향	Frame	DLC	주기	목적
0x200 UltrasonicDistanceCmd	센서 ECU	센서 ECU	→ 판단 ECU	CAN FD	24	10ms	초음파 10개 거리(mm), IMU Yaw(-180~180°), 속도(0.1km/h) 전송
0x100 VehicleControlCmd	판단 ECU	판단 ECU	→ 차량 제어 ECU	CAN	2	10ms	DriveCmd / SteeringCmd 전송 → UART 프레임으로 변환 후 차량 제어보드 송신
0x400 DistanceLevelCmd	판단 ECU	판단 ECU	→ 라즈베리파이	CAN FD	14	100ms	거리 단계 ×10, PCA 상태, 속도, 기어, 긴급 제동 ON/OFF → UI 동기화
0x401 exitStatus	판단 ECU	판단 ECU	→ 라즈베리파이	CAN	1	100ms	자동 출차 진행 상태 (Normal / 진행중 / 완료 / 중단) → 앱 표시
0x201 VehicleStatusCmd	라즈베리파이	라즈베리파이	→ 판단 ECU	CAN	6	12ms	DriveCmd, SteeringCmd, GearStatusCmd, PCA 활성화 여부, 차선 각도
0x300 AutoParkingCmd	라즈베리파이	라즈베리파이	→ 판단 ECU	CAN	1	100ms	자동 출차 시작(직진/좌/우) / 중단 명령 → 긴급 제동 자동 연동



Pygame 기반 블루투스 게임패드

1

조이스틱으로 속도, 조향 값 확인

axis_to_byte() 를 통해
axis(-1.0 ~ 1.0) → uint8(0~255)

2

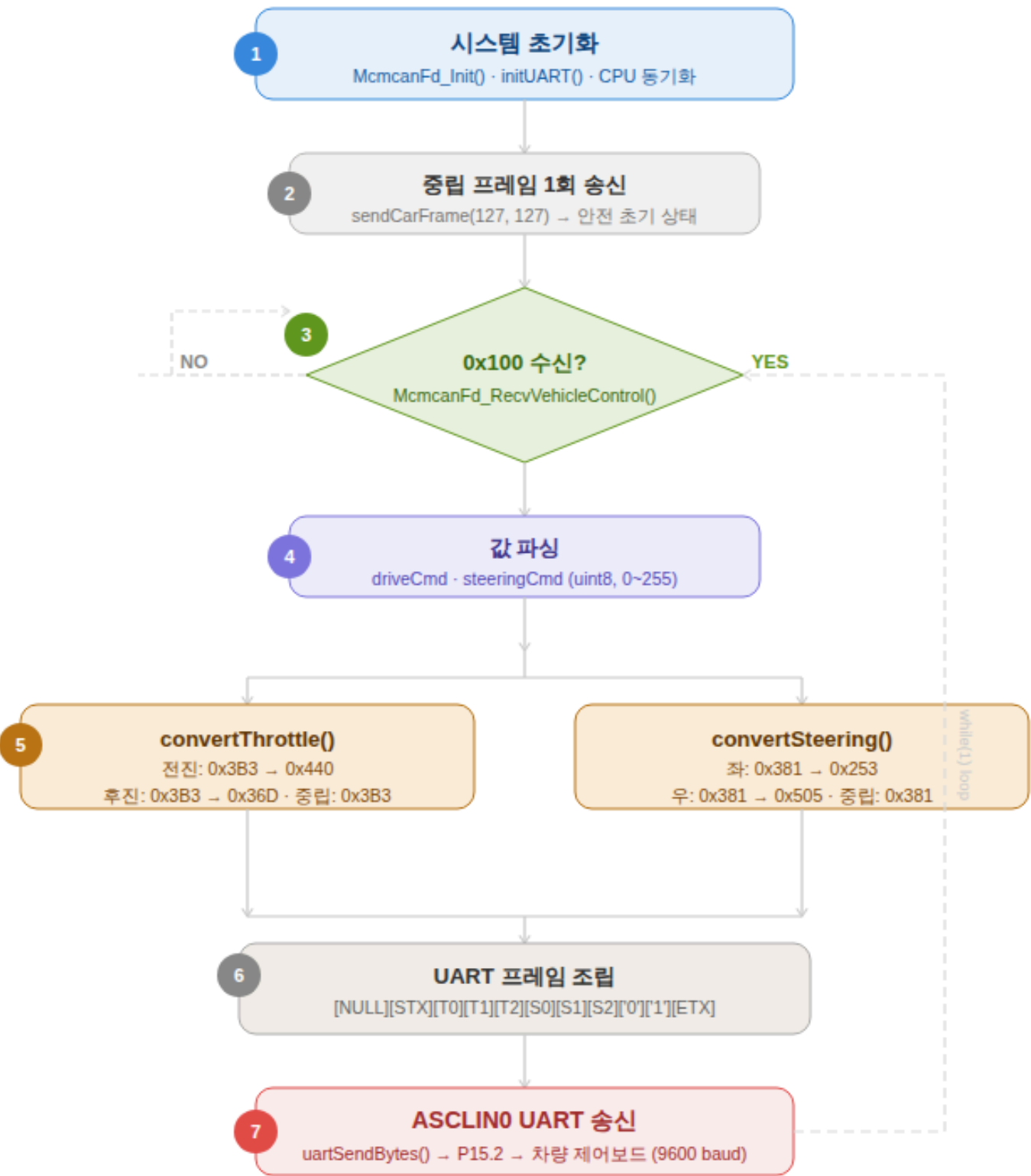
버튼으로 기어 P/D/R 전환, PCA 기능 On/Off

기어 P → 속도, 조향 강제 중립

기어 D → 후진 명령 무시

기어 R → 전진 명령 무시

EmergencyStop == 1 → 속도 조향 강제 중립



기존 차량 제어 Serial Packet을 분석해
[NULL] [STX] [속도 데이터 3 바이트] [조향 데이터 3 바이트] [ETX]
데이터를 보내고 있음을 확인

CAN 0x100의 uint8(0~255)을 파싱

Throttle(0x36D~0x440), Steering(0x253~0x505) 값을 이용해
차량 제어 패킷을 만들어 UART로 송신

시작 시 중립 프레임 먼저 송신
CAN 통신 오류 시 강제 중립 값 송신

센서ECU 초음파 센싱 & RTOS 적용

초음파 거리 기반 충돌 방지 시스템 구현



1 배치 및 역할

- 차량 둘레에 총 10개의 초음파 센서 배치
- 각 센서가 근거리 장애물의 거리·방향 정보를 제공
- 주차, 저속 회피, 협소 공간 진입 시 주변 인지에 활용
- 순차 측정으로 센서 간 교차간섭 최소화



2 GTM TIM 기반 거리 측정

- TC375의 GTM TIM 입력 캡처로 Echo 펄스폭을 하드웨어로 계측
- Rising/Falling edge 기준으로 펄스폭을 us 단위로 계산
- CPU 폴링 대비 타이밍 정밀도 향상 및 측정 부하 감소
- 측정된 Echo 시간으로 거리 변환

$$\text{distance(mm)} \approx \text{echo(us)} \times 0.1715$$

※ GTM TIM 입력 캡처 기반 정밀 시간 측정



3 시스템 처리 흐름



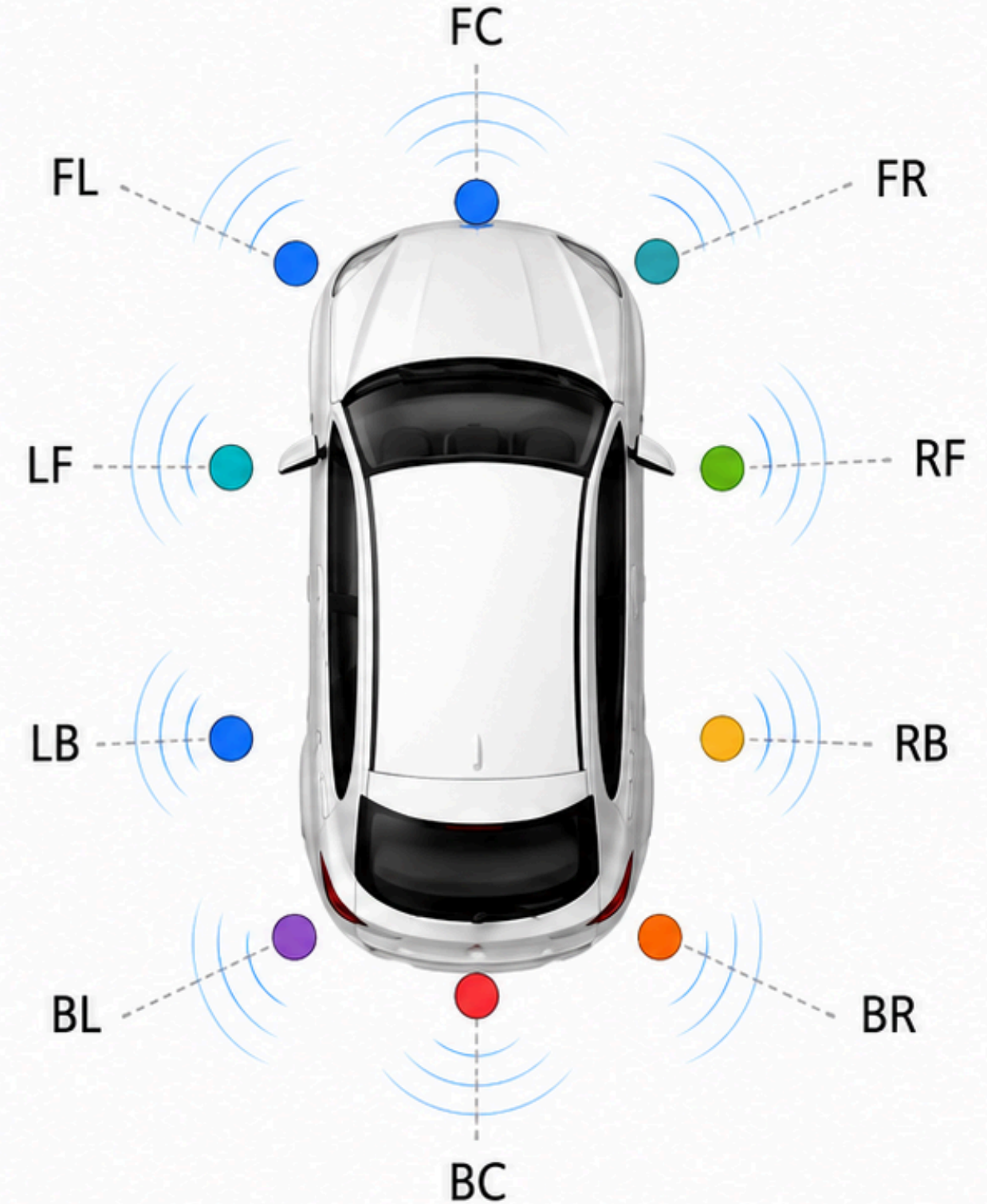
4 필터 적용

- 접근 시에는 큰 가중치 EMA로 빠르게 반영, 이탈 시에는 완만하게 반영
- 큰 거리 증가값은 far-jump gate로 2회 확인 후 반영
- Out-of-range / No echo는 연속 확인 후 상태 갱신
- 100~300us 근접 suspect 값은 연속 확인 후 수용
- Bad measurement 누적 시 마지막 유효값 유지 후 오류 상태 전환

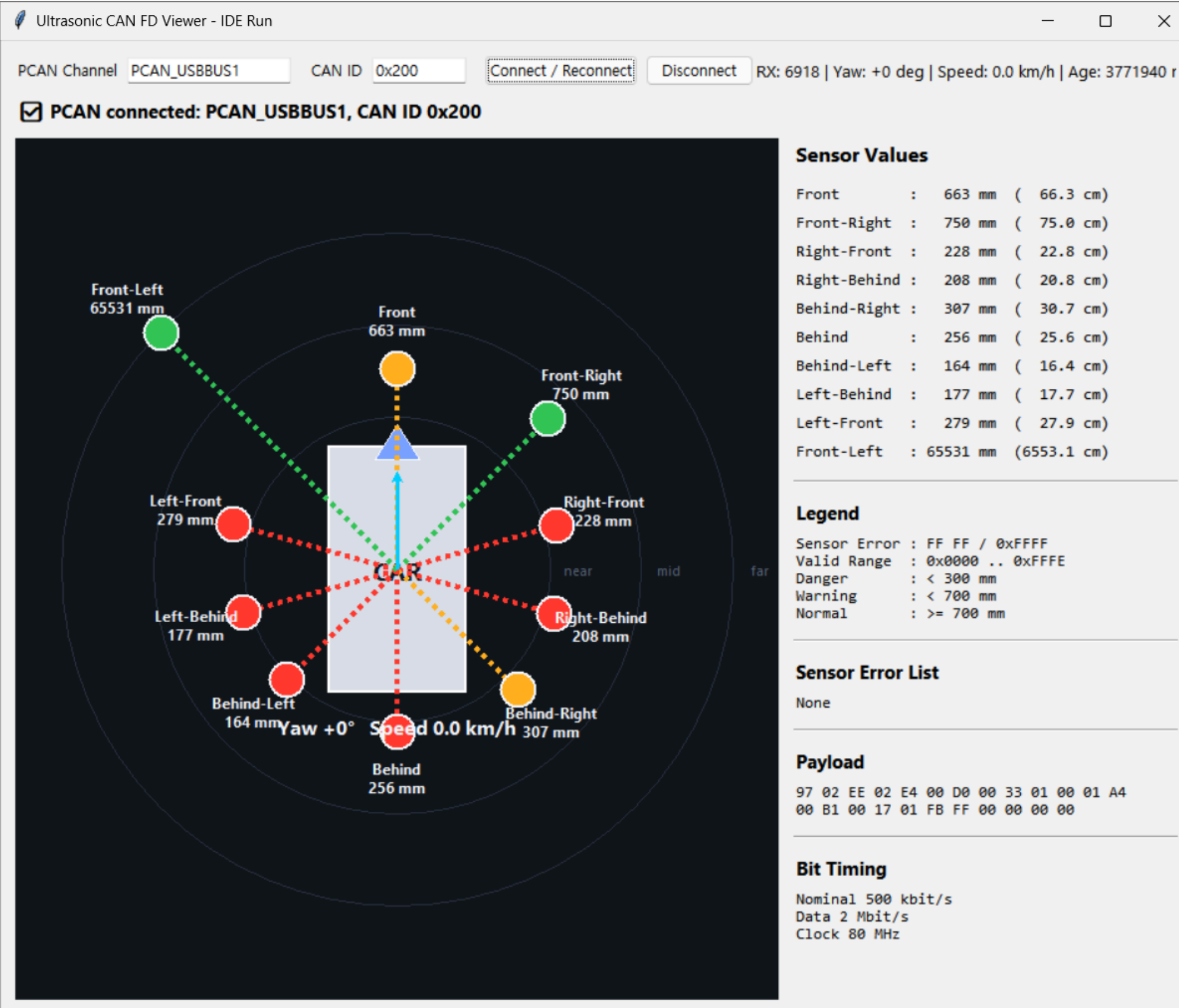


5 한계 및 고려사항

- 잔향(반사파 꼬리)이 다음 측정 슬롯에 영향 가능
- 측면·모서리에서는 다중 반사로 오차 증가 가능
- 흡음 재질, 경사진 물체, 매우 가까운 물체는 측정 불안정
- 고속 주행보다 저속 주차·근접 감지에 적합

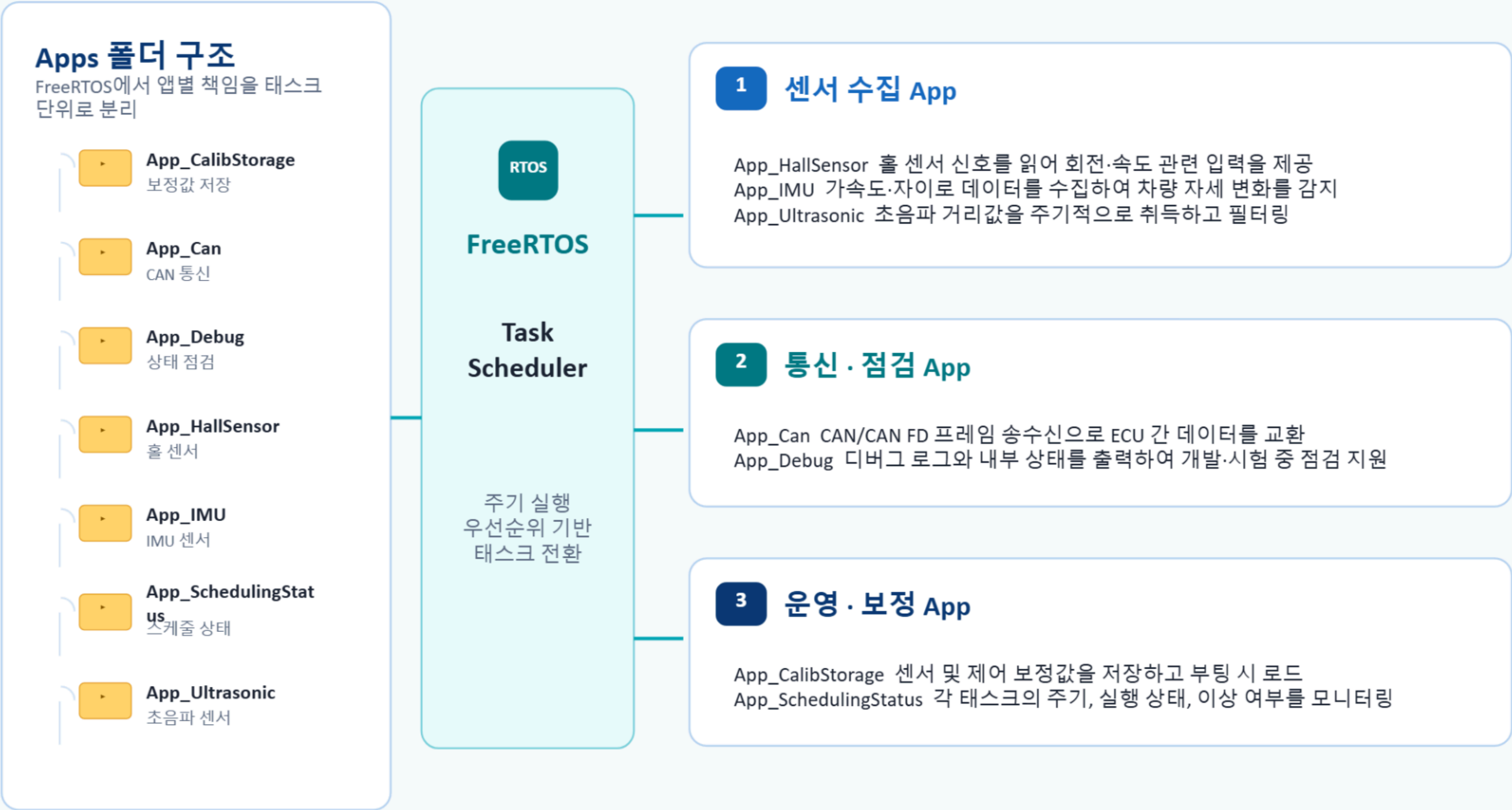


초음파 센서



센서 디버깅용 프로그램을 파이썬으로 제작,
디버깅에 활용

센서 필터 로직과 순차발사 로직을
이 프로그램을 통해 고도화



구조 요약: 기능별 App이 FreeRTOS 태스크로 독립 실행되고, 공유 데이터와 CAN 메시지를 통해 전체 시스템 흐름에 연결됨

판단 ECU 충돌 방지 경고

초음파 거리 기반 충돌 방지 시스템 구현

App 계층:

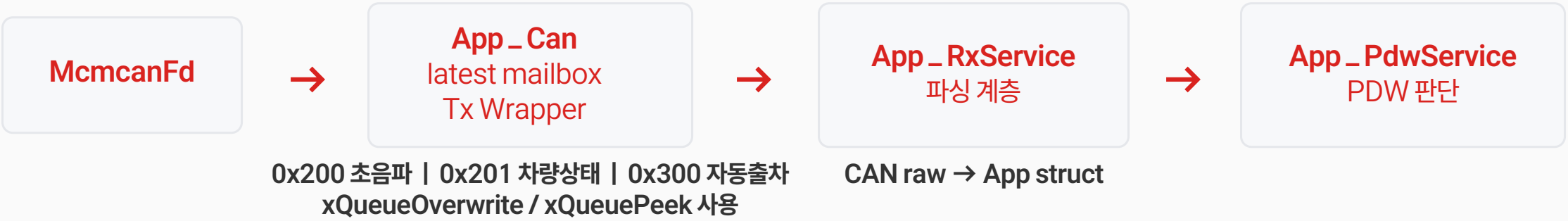
App_AutoExitService
App_Can
App_DriveService
App_PdwService
App_RxService
App_StatusTxService

Driver 계층:

CanMsg.h
McmcanFd.c
McmcanFd.h
McmcanFd_Cfg.h

내부 구현 구조

CAN 수신값을 바로 판단에 섞지 않고, 최신값 관리와 파싱 계층을 분리했습니다.



RTOS Task 주기 설계

수신, 변환, 판단, 송신의 책임을 나눠 기능 추가 시 영향 범위를 줄이고 RTOS task 단위로 디버깅할 수 있게 했습니다.

CAN Rx	1ms	수신 프레임을 최신 mailbox에 반영
PDW Service	10ms	거리값을 위험도 level로 판단
Status Tx	10ms	0x400 상태를 HMI/부저로 전달
Drive Service	12ms	최종 0x100 제어 명령 송신

PDW 판단 로직과 경고 출력

거리 입력과 차량 상태 조건을 바탕으로 5단계 위험도를 생성

입력

초음파 거리값 10개
각 방향별 거리 [mm]
Gear / PDW Switch / 속도
자동출차 명령 상태

활성 조건

Gear P이면 비활성
PDW Switch Off이면 비활성
속도 3 초과이면 비활성
자동출차 명령 중이라면 비활성

출력

방향별 PDW level
PDW Activated
emergencyStop
CAN 0x400 DistanceLevel

NO OBSTACLE

0mm 또는 1000mm 초과

SAFE

≤ 1000mm

CAUTION

≤ 300mm

NEAR

≤ 250mm

DANGER

≤ 200mm

emergencyStop 생성

거리값을 단계별 위험도로 변환하고, HMI/부저가 사용할 0x400 상태 메시지로 송신

PCA-HMI 차량 디스플레이 구현

차량과 사용자 사이에서 정보를 시각화하는 HMI

01

차량 상태와 주차 보조 정보를
운전자에게 시각적으로 제공

02

속도, 기어, 충돌방지 상태
장애물 감지, 후방 카메라 화면 표시

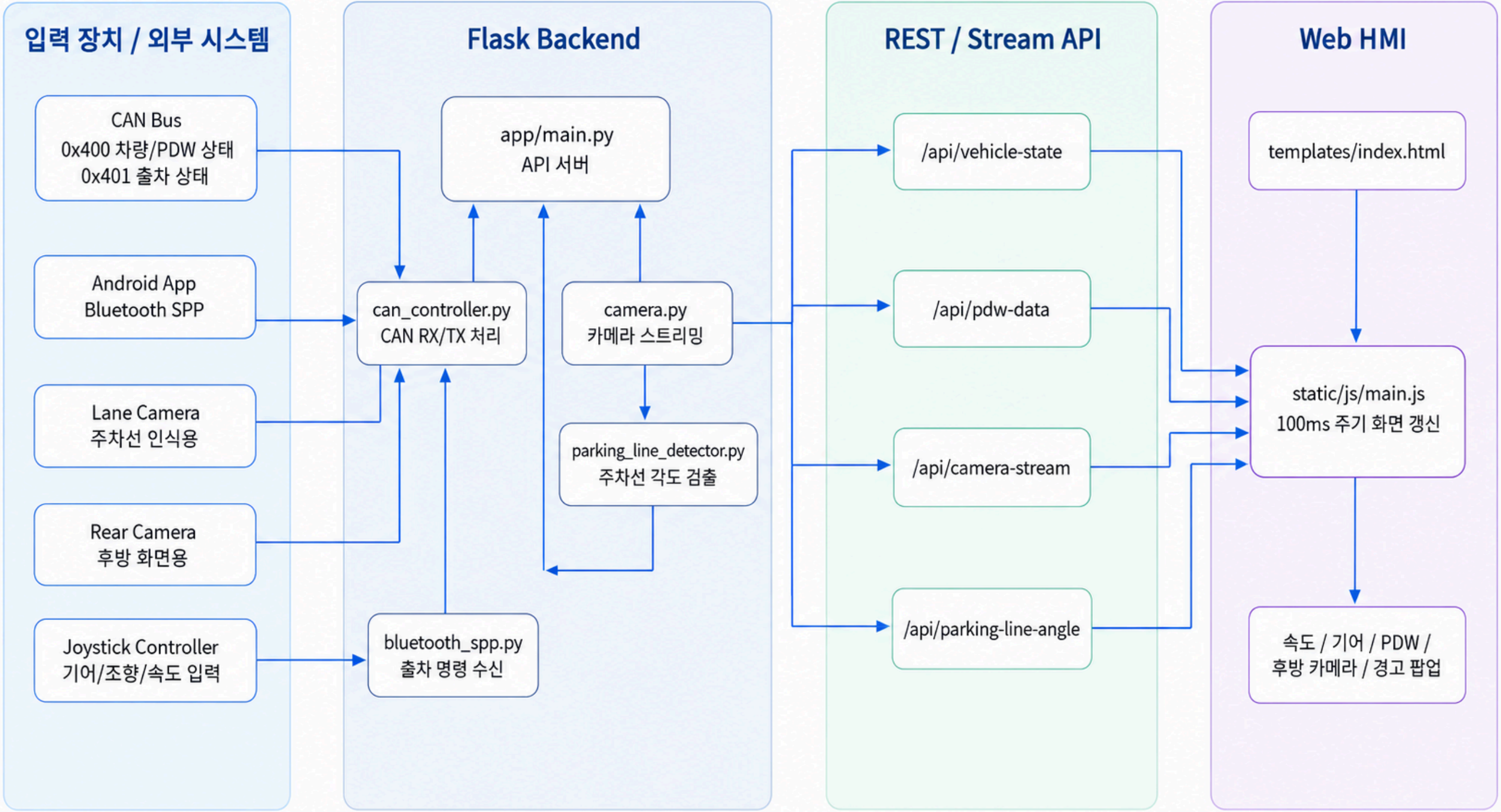
03

CAN, 블루투스, 카메라 데이터를
Flask 서버에서 통합 처리

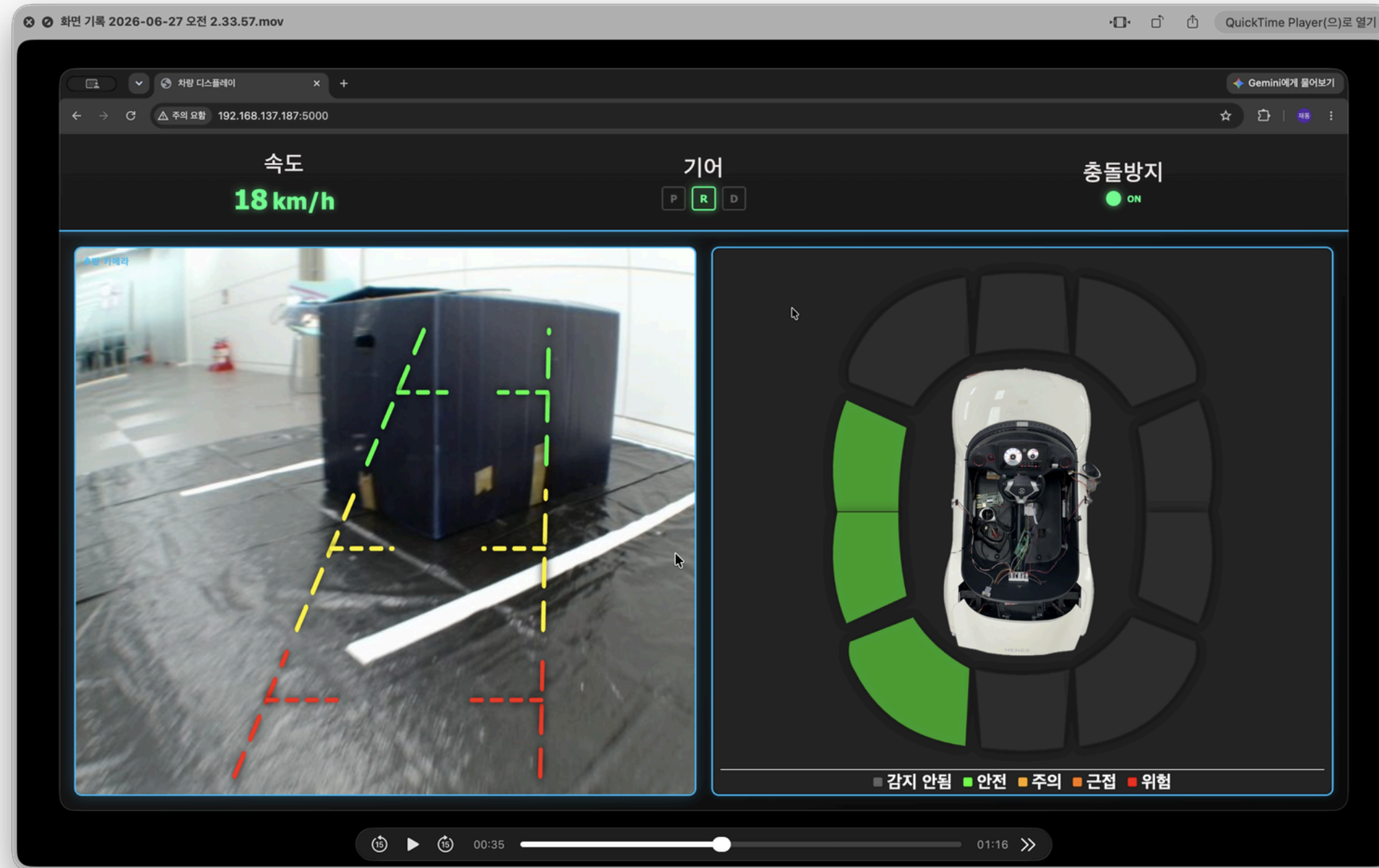
04

웹 기반 HMI로
실시간 차량 정보를 표현

HMI 서버 아키텍처



HMI 상세기능 - 차량 상태 실시간 표시



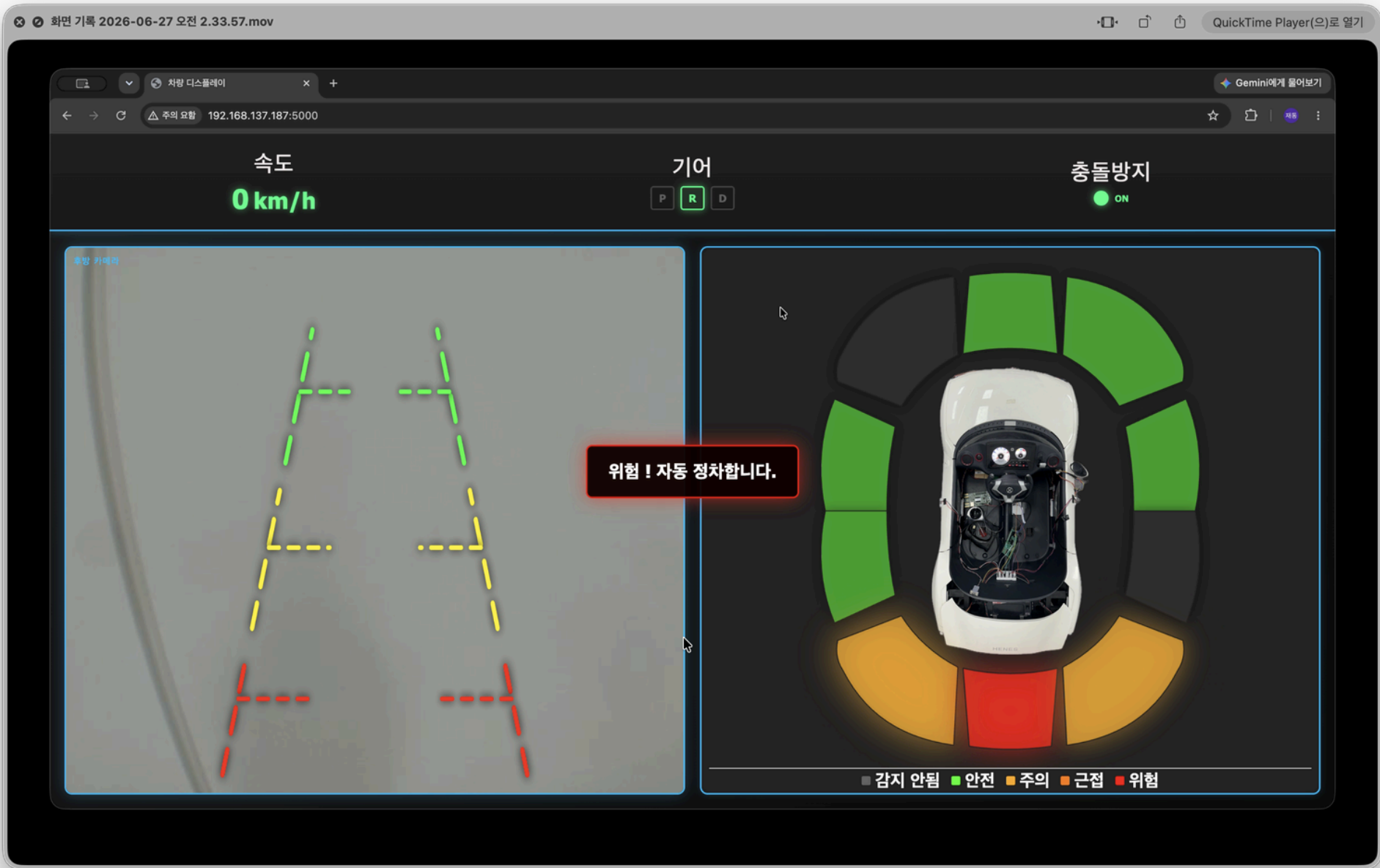
/api/vehicle-state를 통해 차량 상태 데이터 수신

속도, 기어 상태, 충돌방지 기능 상태 표시

Emergency Stop 신호 수신 시 경고 팝업 출력

CAN 수신 데이터를 사용자에게 직관적인 UI로 변환

HMI 상세기능 - PDW 장애물 감지 표시



CAN 0x400 메시지에서 10방향 장애물 레벨 수신

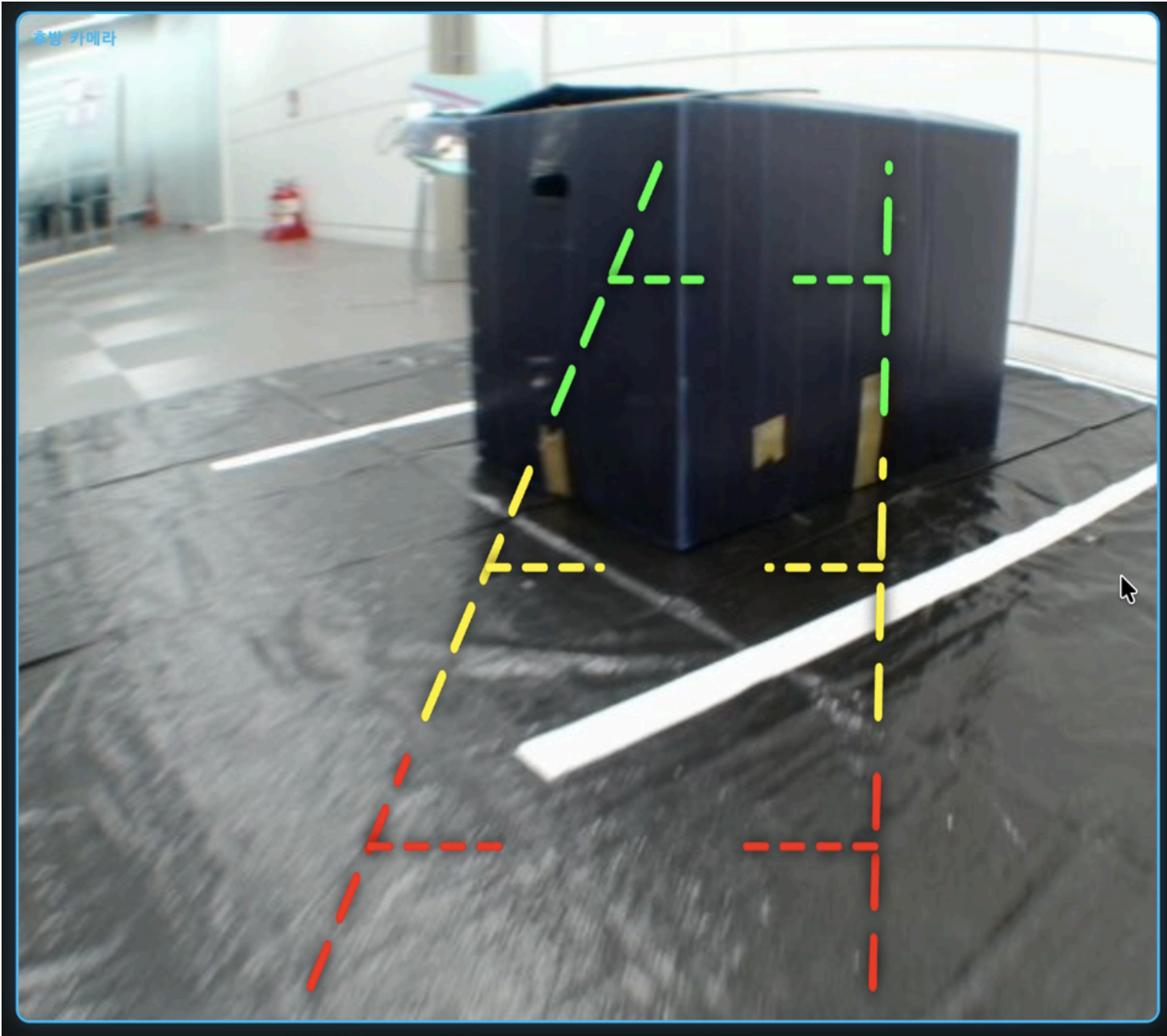
차량 주변 영역을 방향별로 나누어 감지 상태 표시

장애물 위험도에 따라 색상 단계 적용

운전자가 어느 방향에 장애물이 있는지 즉시 확인 가능



HMI 상세기능 - 후방 카메라와 조향 가이드라인



R 기어 상태에서 후방 카메라 스트림 자동 활성화

/api/camera-stream으로 MJPEG 영상 제공

조향각 데이터를 기반으로 후방 가이드라인 동적 표시

주차 시 예상 이동 방향을 시각적으로 안내

HMI 상세기능 - 주차선 인식 로직



카메라 영상에서 흰색 주차선 후보 추출

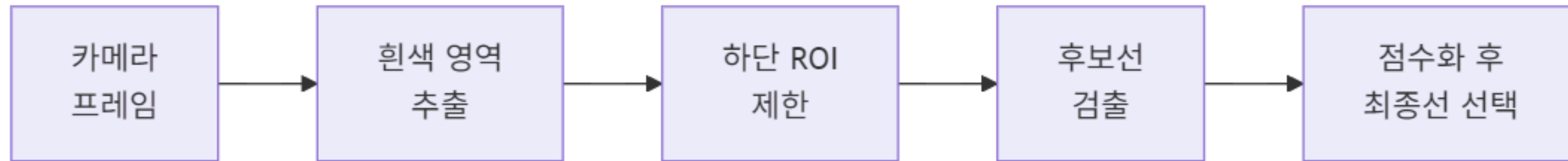
Canny Edge로 경계 검출

HoughLinesP로 직선 후보 검출

최종 주차선의 기울기로 차량 기준 각도 계산

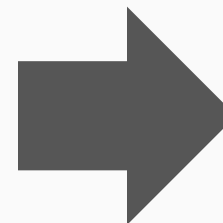
문제점

Canny와 HoughLinesP 기본 검출 시 바닥 반사광까지 선으로 인식했고
주차선이 아닌 선 후보가 과도하게 검출됨



개선 방법

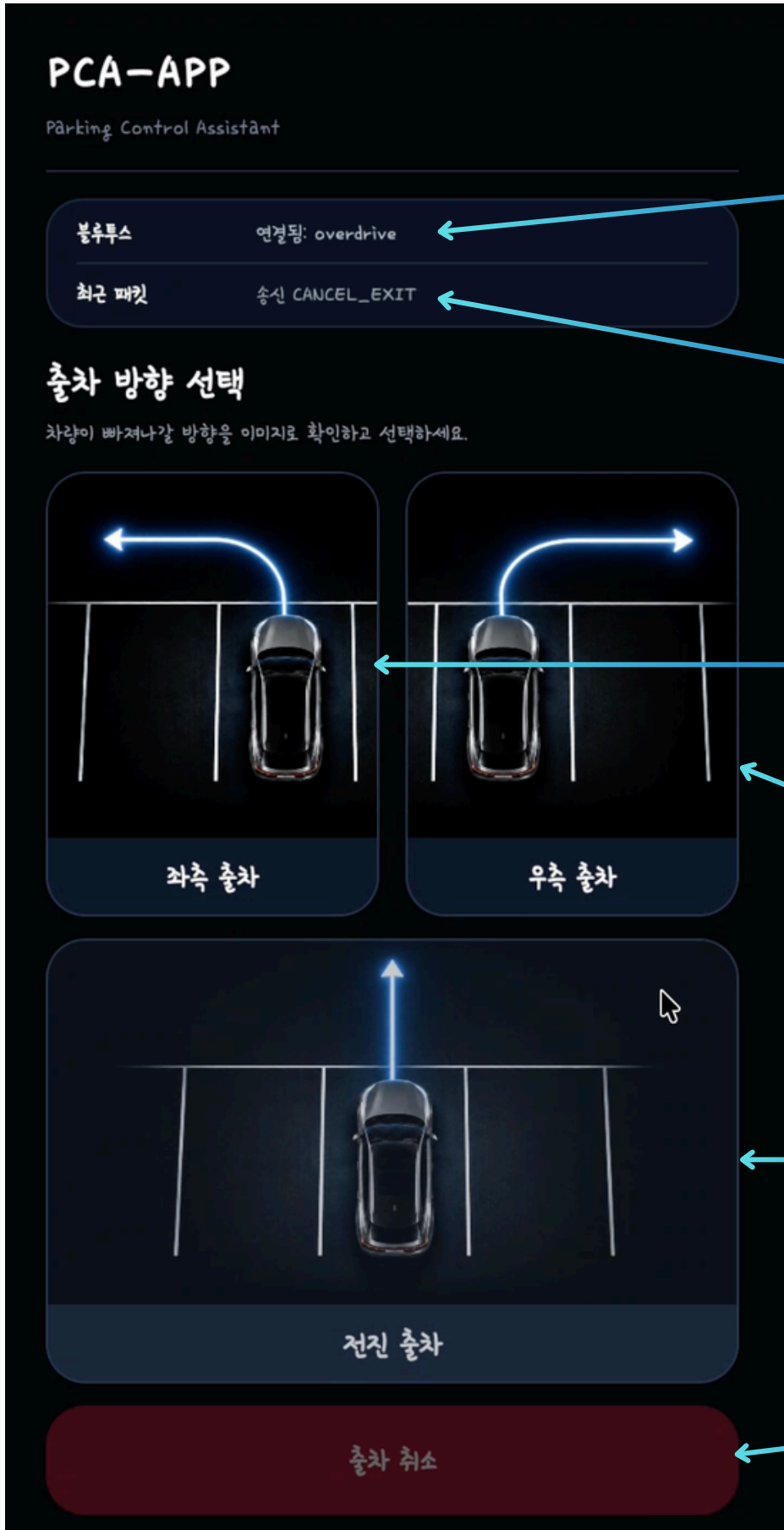
- 흰색 영역 추출 + 하단 ROI 제한
- Morphology로 노이즈 제거
- 길이, 두께 비율, 각도 기준으로 후보 필터링



**반사광으로 인해 40개 이상 검출되던 선 후보를
필터링 후 1~3개의 주차선 후보로 축소**

PCA-APP Android Application

원격으로 차량 출차 제어 기능을 제공



Bluetooth 연결 상태

연결된 차량 이름 및 연결 상태 확인

최근 패킷 정보

가장 최근에 송/수신된 패킷 정보를 확인

좌측 출차 버튼

좌측 출차 실행

우측 출차 버튼

우측 출차 실행

전진 출차 버튼

전진 출차 실행

출차 취소 버튼

출차 진행 중에만 활성화되며, 진행 중인 출차 즉시 취소

자동 출차

사용자 원격 제어로 자동으로 출차되는 기능

IEEE Xplore[®]

Browse

My Settings

Help

Institutional Sign In

All

ADVANCED SEARCH

Conferences > 2025 International Technical ...

A Precise Autonomous Parking System Using Ultrasonic Sensors

Publisher: IEEE

Cite This

PDF

[Hyo Jin Park](#)

[Hyeong Keun Hong](#)

[Yeong Gwang Choi](#)

[Eun Ho Kim](#)

[Young Hoon Suh](#)

[Jae Wook Jeon](#)

All Authors

1

Cites in Paper

185

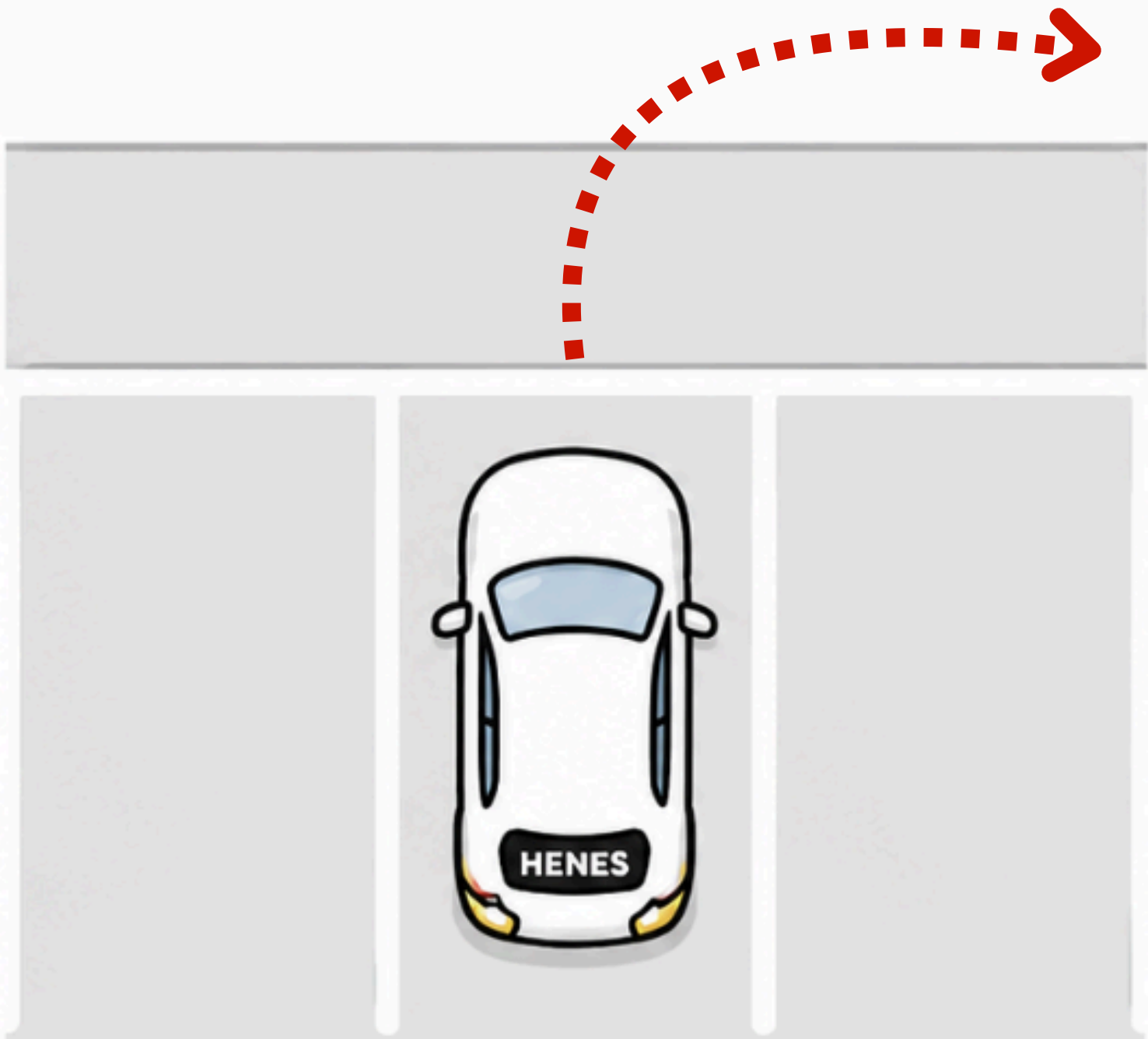
Full Text Views

R

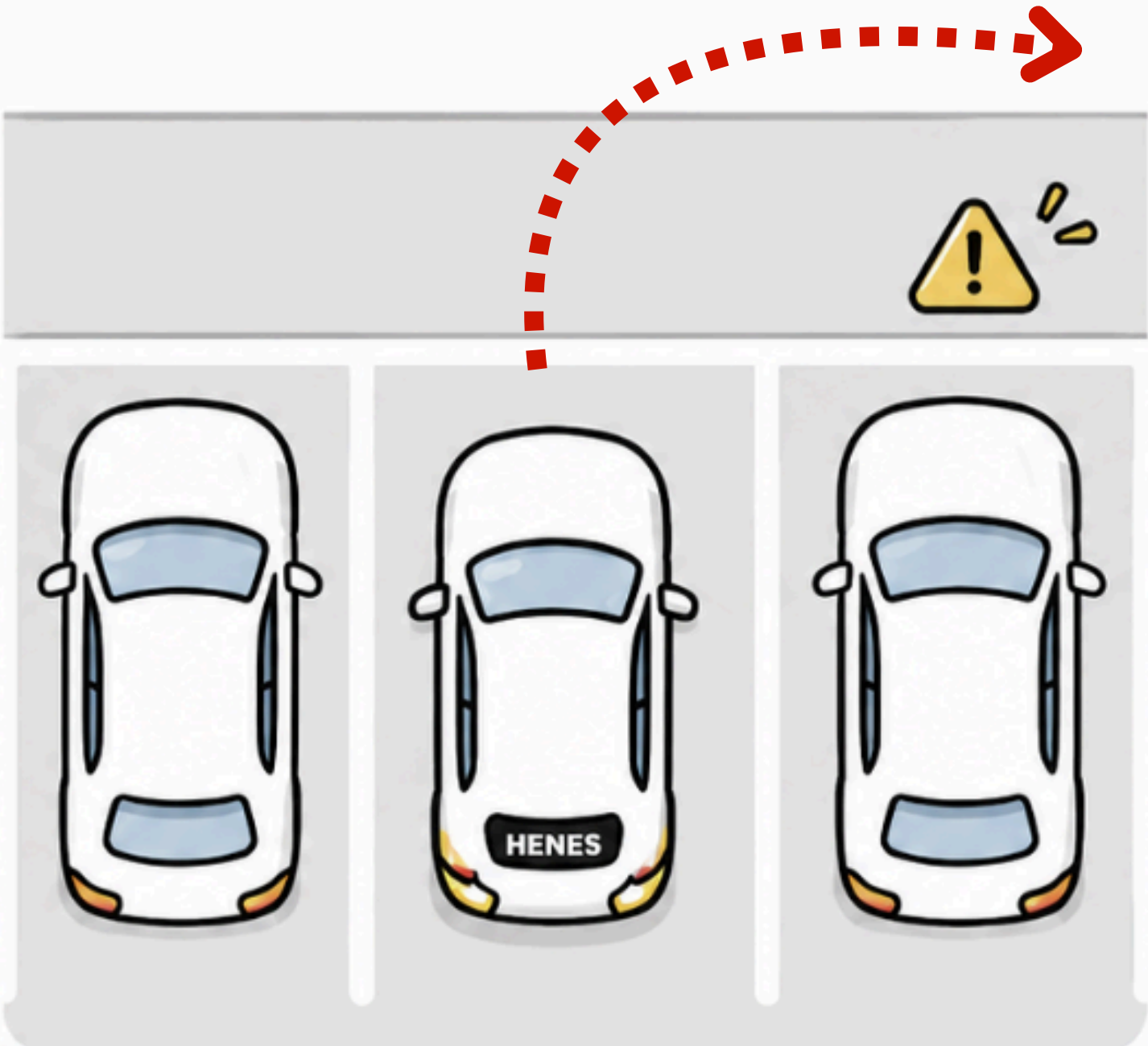
©

논문 링크 : <https://ieeexplore.ieee.org/document/11137729>

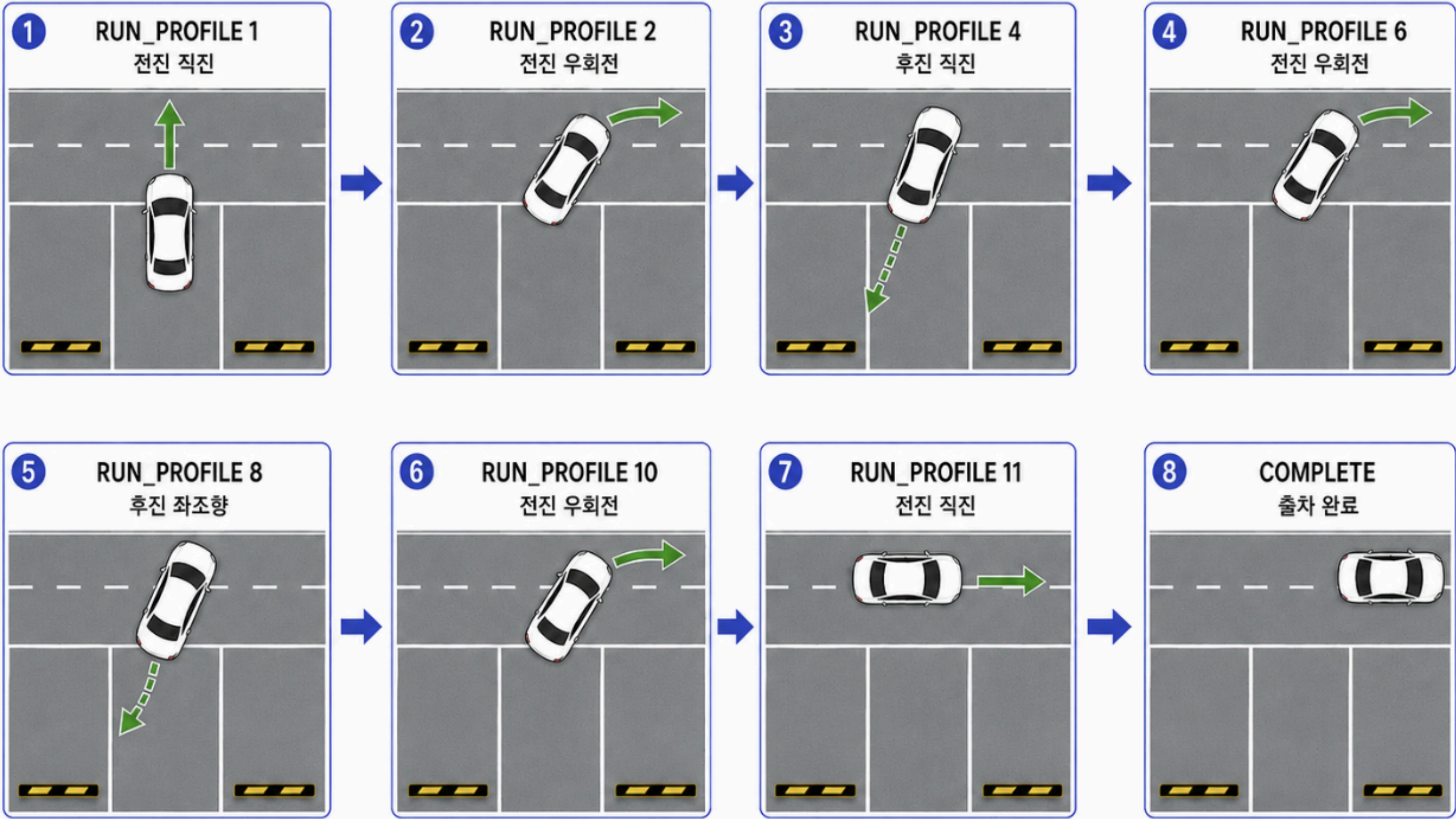
01



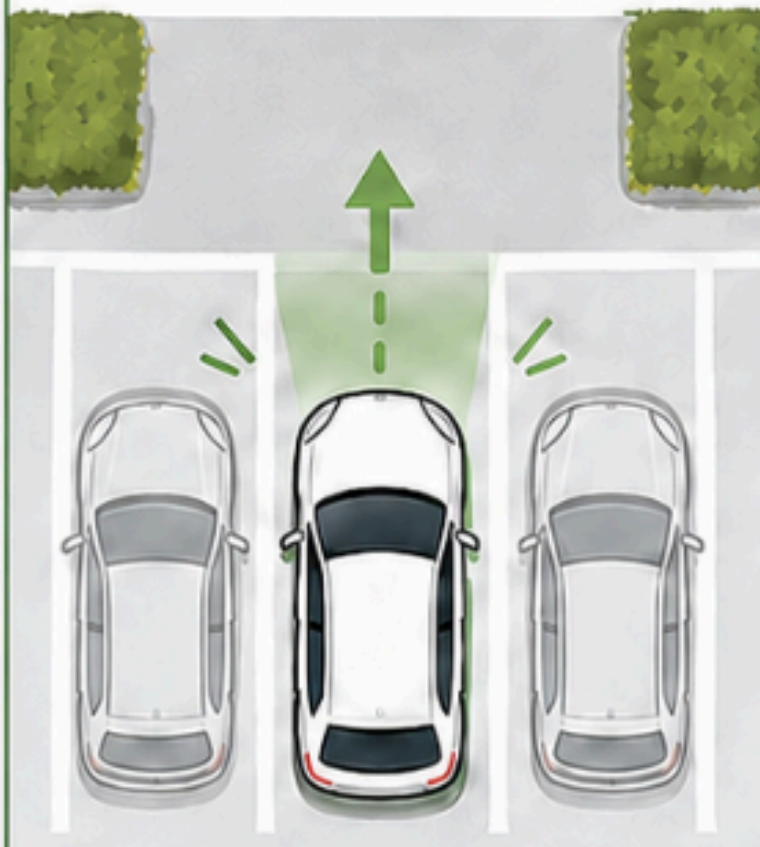
02



FSM

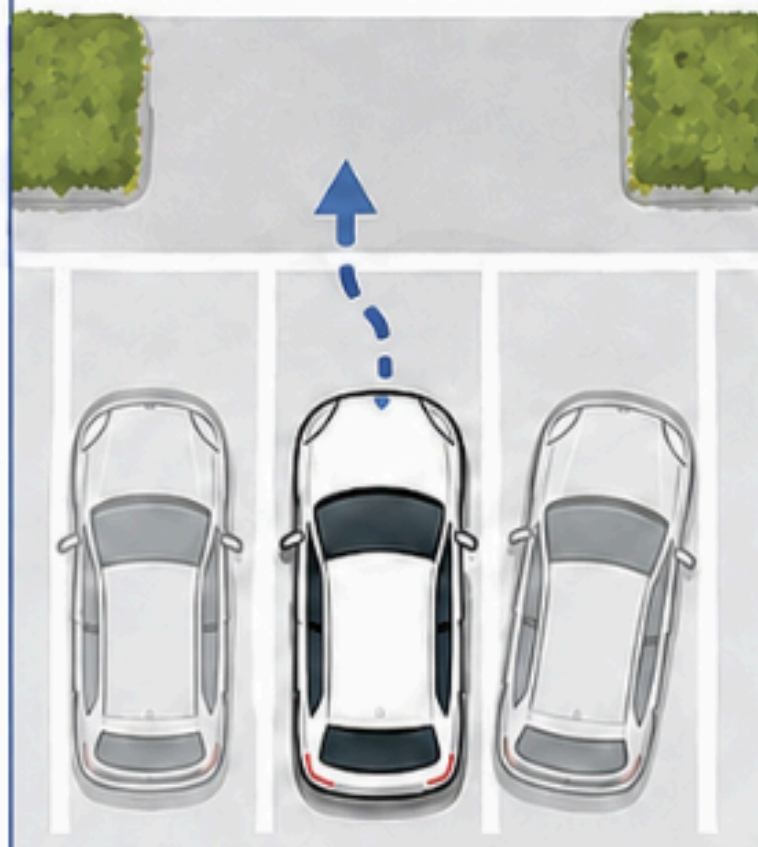


1 회피 없음 (NORMAL)



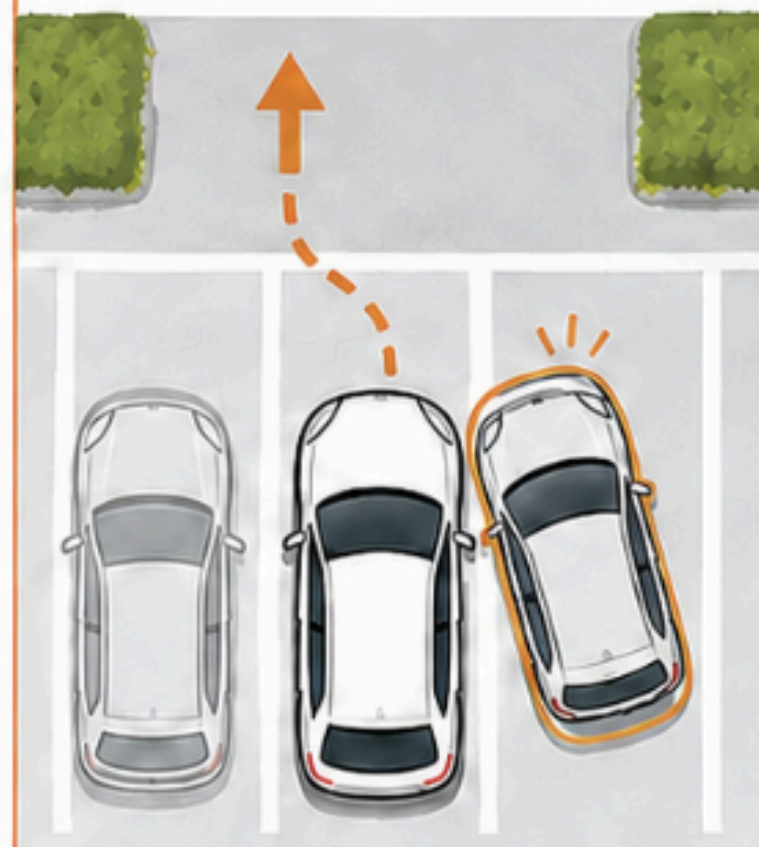
- ✓ 나가려는 방향이 충분히 넓은
 - ✓ 앞쪽도 뒤쪽도 여유 있음
 - ✓ 기울기 영향 거의 없음
- 회피 없이 바로 출차

2 짧게 회피 (SHORT)



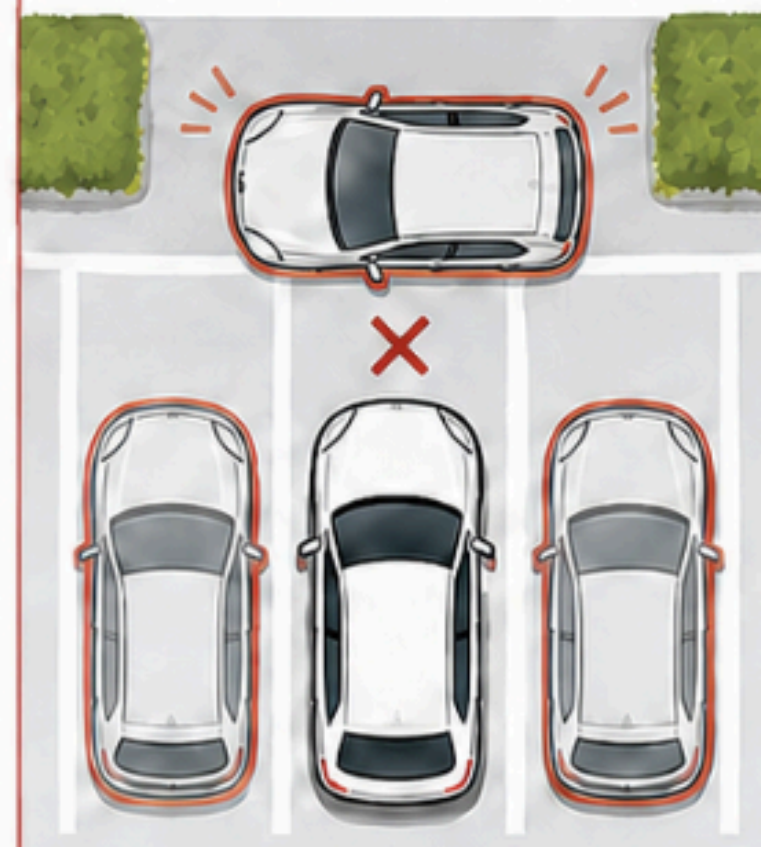
- ✓ 뒤쪽이 더 가까움
 - ✓ 앞쪽은 비교적 여유 있음
 - ✓ 조금만 피하면 출차 가능
- 짧게 회피 후 출차

3 길게 회피 (LONG)



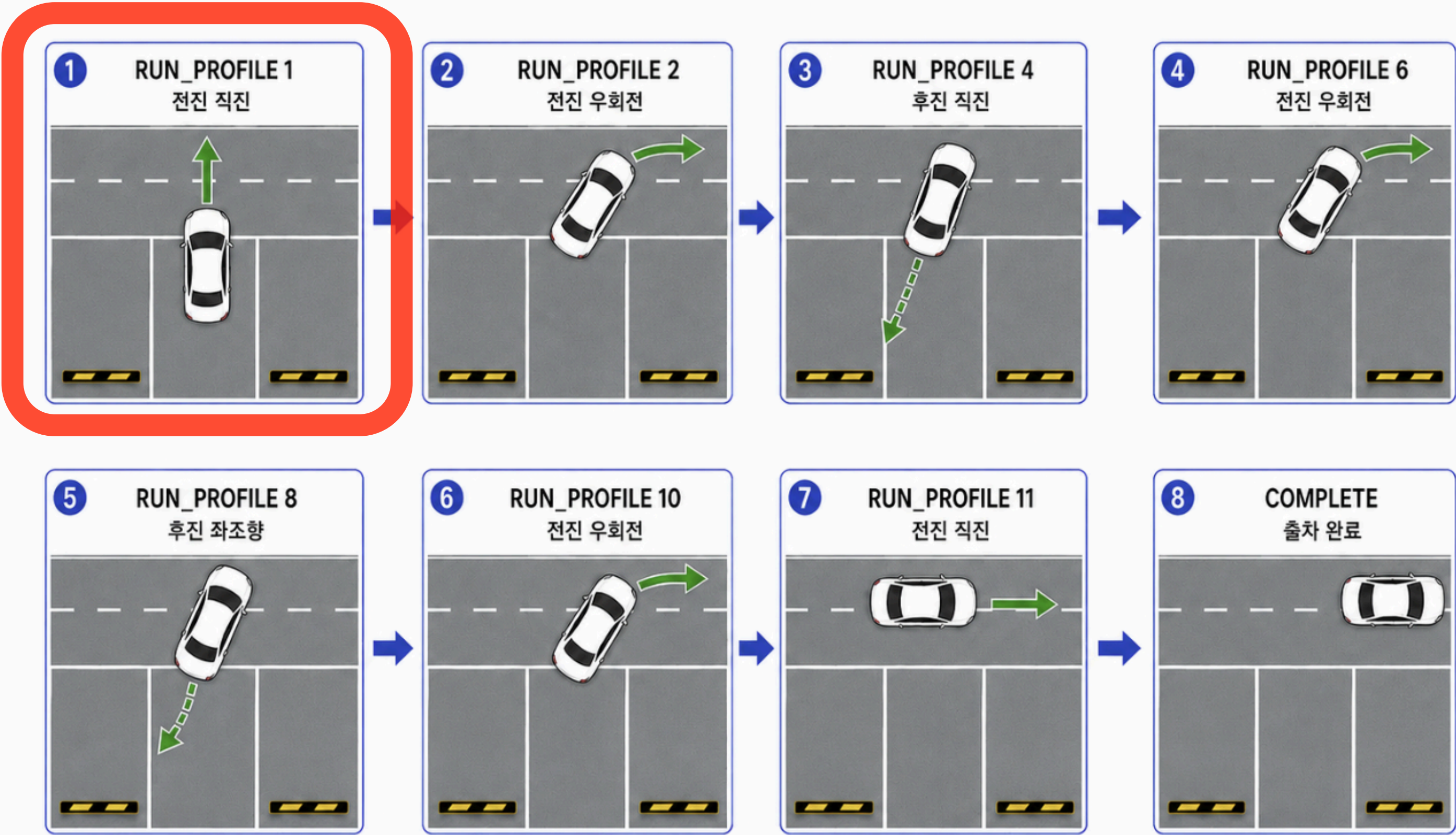
- ✓ 앞쪽이 더 가까움
 - ✓ 차가 기울어져 앞쪽 간섭이 큼
 - ✓ 전체적으로 좁게 느껴짐
 - ✓ 더 크게 피해야 안전함
- 길게 회피 후 출차

4 출차 불가 (BLOCKED)



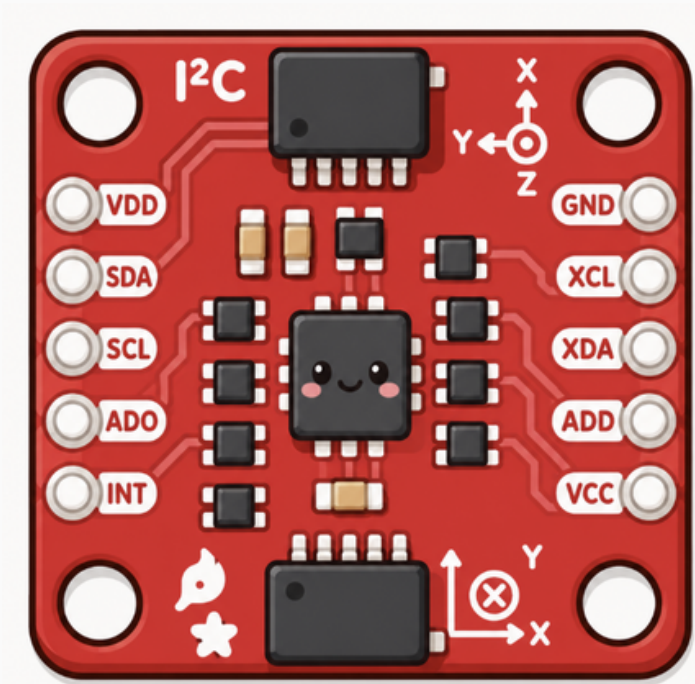
- ✓ 출차 방향이 막혀 있음
 - ✓ 반대쪽으로 피할 공간도 부족함
 - ✓ 안전하게 회피하기 어려움
- 출차 중단

FSM

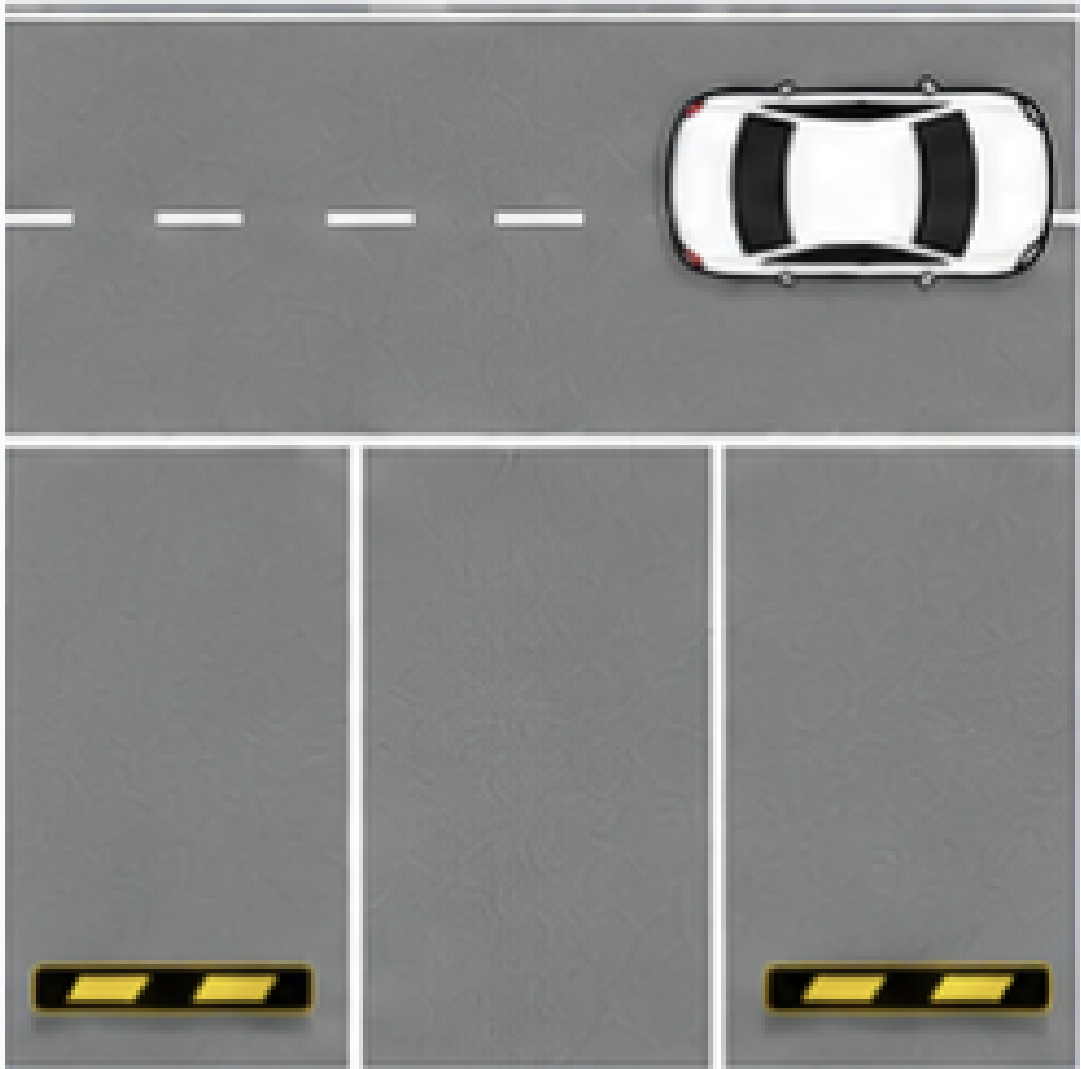
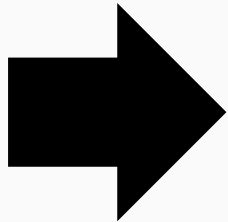




내 차 각도



YAW



90도

시연 영상

이재필

세번째 프로젝트다보니 팀워크가 점차 좋아지는 걸 느꼈습니다. 언제나 열정적이고 늘 팀을 우선적으로 생각해주는 팀원들을 만나서 팀장으로서 너무 축복받았다고 생각하고 감사했습니다. 열심히 달렸던만큼 다들 정상에서 만나기를 소망합니다.

김병현

마지막 프로젝트라 전체 일정이 상대적으로 저번 프로젝트보다 순조로웠던 것 같습니다. 그리고 결과도 기대한 대로 나와서 만족스럽고 같이 고생한 팀원들에게 감사드립니다.

황선안

마지막 프로젝트라 다들 대충하고 넘기지 않을까 걱정했으나 그 누구보다 열심히 참여해준 팀원분들께 감사인사 드립니다. 문제가 생겨도 금방 해결할 수 있다는 용기를 얻었습니다.

김재동

마지막까지 잘 마무리할 수 있어서 뿌듯했습니다. 블루투스 통신이나 출차 동작 처리처럼 생각보다 신경 써야 할 부분이 많다는 것을 느꼈습니다. 팀원들과 함께 문제를 해결하면서 많은 것을 배울 수 있었고, 끝까지 함께 고생해준 팀원들에게 감사합니다.

김건우

지금까지 프로젝트를 진행하면서 정말 힘들었는데 마지막 프로젝트까지 잘 끝낸것 같아 기분이 정말 좋습니다. 끝까지 힘들었지만 각자 맡은 역할 잘 수행해준 팀원들에게 정말고맙습니다!

이소연

프로젝트를 진행할 때마다 통합이 순탄한 적이 한번도 없었는데, 이번에는 그래도 지난 두 번 프로젝트의 경험이 있어 디버깅하는데 시간이 비교적 덜 걸렸던 것 같습니다. 무엇보다 세 번의 프로젝트를 함께하며 어려운 상황에서도 항상 긍정적인 태도로 끝까지 함께해 준 팀원들에게 많은 것을 배웠습니다.

<https://github.com/PCA-Overdrive>

Thank You

Overdrive

김건우, 김재동, 김병현, 이소연, 이재필, 황선안